

Oscillations of a Driven Triple Pendulum

Parth Bhargava · A0310667E

Soham Bhar · A0311156R

PC3236 Computational Methods in Physics

March 18, 2026

1 Abstract

The chaotic dynamics of a driven triple pendulum are investigated numerically. The equations of motion are derived from the Lagrangian for three coupled point masses on rigid, massless rods, with a periodic torque $\tau(t) = \tau_0 \cos(\omega_d t)$ applied at the pivot of the first pendulum. The resulting system of three coupled second-order ODEs is reformulated as six first-order equations and integrated using a custom fourth-order Runge-Kutta (RK4) scheme with time step $\Delta t = 10^{-3}$ s. The simulation produces time series, phase portraits, Poincaré sections, energy evolution, and sensitivity-to-initial-conditions plots that together demonstrate the transition from regular to chaotic motion. A bifurcation diagram over a range of driving amplitudes reveals the onset of chaos. The custom RK4 implementation is validated against scipy's adaptive RK45 solver, with agreement to $< 10^{-6}$ rad over 20 s of integration.

2 Description of the Problem

The simple pendulum is one of the most studied systems in classical mechanics, exhibiting clean periodic oscillations for small angles and mild nonlinearity for large swings. Coupling two pendulums already introduces the possibility of chaotic behaviour, and adding a third greatly enriches the dynamics: the phase space becomes six-dimensional, and even small perturbations in initial conditions can lead to exponentially diverging trajectories.

When an external periodic driving force is added, the system acquires an energy source that can push it through a sequence of bifurcations, from simple periodic motion at low driving amplitude, through period-doubling cascades, to fully developed chaos at stronger driving. This makes the driven triple pendulum a useful model for studying fundamental concepts in nonlinear dynamics: Lyapunov divergence, strange attractors, and the boundary between regular and chaotic regimes.

The problem studied here is: given three point masses m_1, m_2, m_3 connected by rigid rods of lengths L_1, L_2, L_3 , with a periodic torque $\tau_0 \cos(\omega_d t)$ applied at the first pivot, determine the angular trajectories $\theta_1(t), \theta_2(t), \theta_3(t)$ and characterise the system's dynamical behaviour as the driving amplitude varies.

3 Equations to be Solved

3.1 Coordinates and Kinematics

Let θ_i be the angle of the i -th rod measured from the downward vertical. The Cartesian positions of the three bobs are:

$$x_1 = L_1 \sin \theta_1, \quad y_1 = -L_1 \cos \theta_1 \quad (1)$$

$$x_2 = x_1 + L_2 \sin \theta_2, \quad y_2 = y_1 - L_2 \cos \theta_2 \quad (2)$$

$$x_3 = x_2 + L_3 \sin \theta_3, \quad y_3 = y_2 - L_3 \cos \theta_3 \quad (3)$$

The velocities follow by differentiation:

$$\dot{x}_1 = L_1 \dot{\theta}_1 \cos \theta_1, \quad \dot{y}_1 = L_1 \dot{\theta}_1 \sin \theta_1 \quad (4)$$

and similarly for bobs 2 and 3, each inheriting the velocity of its parent joint.

3.2 Lagrangian

The kinetic energy $T = \frac{1}{2} \sum_i m_i (\dot{x}_i^2 + \dot{y}_i^2)$ and potential energy $V = \sum_i m_i g y_i$ give the Lagrangian $\mathcal{L} = T - V$. The Euler-Lagrange equations $\frac{d}{dt} \frac{\partial \mathcal{L}}{\partial \dot{\theta}_i} - \frac{\partial \mathcal{L}}{\partial \theta_i} = Q_i$ (where Q_i is the generalised force for the i -th coordinate) yield a matrix equation:

$$\mathbf{M}(\boldsymbol{\theta}) \ddot{\boldsymbol{\theta}} = \mathbf{F}(\boldsymbol{\theta}, \dot{\boldsymbol{\theta}}, t) \quad (5)$$

3.3 Mass Matrix

The 3×3 mass matrix $\mathbf{M}$ has elements:

$$M_{11} = (m_1 + m_2 + m_3) L_1^2 \quad (6)$$

$$M_{12} = M_{21} = (m_2 + m_3) L_1 L_2 \cos(\theta_1 - \theta_2) \quad (7)$$

$$M_{13} = M_{31} = m_3 L_1 L_3 \cos(\theta_1 - \theta_3) \quad (8)$$

$$M_{22} = (m_2 + m_3) L_2^2 \quad (9)$$

$$M_{23} = M_{32} = m_3 L_2 L_3 \cos(\theta_2 - \theta_3) \quad (10)$$

$$M_{33} = m_3 L_3^2 \quad (11)$$

3.4 Forcing Vector

The forcing vector $\mathbf{F}$ contains the Coriolis-like coupling terms, gravitational torques, and the external drive:

$$F_1 = (m_2 + m_3) L_1 L_2 \dot{\theta}_2^2 \sin(\theta_1 - \theta_2) + m_3 L_1 L_3 \dot{\theta}_3^2 \sin(\theta_1 - \theta_3) - (m_1 + m_2 + m_3) g L_1 \sin \theta_1 + \tau_0 \cos(\theta_1) \quad (12)$$

$$F_2 = -(m_2 + m_3) L_1 L_2 \dot{\theta}_1^2 \sin(\theta_1 - \theta_2) + m_3 L_2 L_3 \dot{\theta}_3^2 \sin(\theta_2 - \theta_3) - (m_2 + m_3) g L_2 \sin \theta_2 \quad (13)$$

$$F_3 = -m_3 L_1 L_3 \dot{\theta}_1^2 \sin(\theta_1 - \theta_3) - m_3 L_2 L_3 \dot{\theta}_2^2 \sin(\theta_2 - \theta_3) - m_3 g L_3 \sin \theta_3 \quad (14)$$

3.5 State-Space Formulation

Defining the state vector $\mathbf{y} = (\theta_1, \theta_2, \theta_3, \omega_1, \omega_2, \omega_3)^T$ where $\omega_i = \dot{\theta}_i$, the system becomes six first-order ODEs:

$$\dot{\theta}_i = \omega_i, \quad \dot{\omega}_i = [\mathbf{M}^{-1} \mathbf{F}]_i \quad (15)$$

This is the form integrated numerically.

3.6 Fourth-Order Runge-Kutta Method

Given the ODE $\dot{\mathbf{y}} = \mathbf{f}(t, \mathbf{y})$, a single RK4 step from t_n to $t_{n+1} = t_n + h$ is:

$$\mathbf{k}_1 = \mathbf{f}(t_n, \mathbf{y}_n) \quad (16)$$

$$\mathbf{k}_2 = \mathbf{f}\left(t_n + \frac{h}{2}, \mathbf{y}_n + \frac{h}{2} \mathbf{k}_1\right) \quad (17)$$

$$\mathbf{k}_3 = \mathbf{f}\left(t_n + \frac{h}{2}, \mathbf{y}_n + \frac{h}{2} \mathbf{k}_2\right) \quad (18)$$

$$\mathbf{k}_4 = \mathbf{f}(t_n + h, \mathbf{y}_n + h \mathbf{k}_3) \quad (19)$$

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \frac{h}{6} (\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4) \quad (20)$$

The local truncation error is $O(h^5)$, giving global error $O(h^4)$.

4 Description of Computational Methodology and Implementation

4.1 Overview

The simulation is implemented in a single Python script (`triple_pendulum.py`, 270 lines). The structure is:

1. Define physical parameters ($g, L_i, m_i, \tau_0, \omega_d$) as module-level constants.
2. Implement `mass_matrix( $\theta$ )` and `forcing( $\theta, \omega, t$ )` as separate functions, each returning NumPy arrays.
3. The function `derivs(t, y)` computes $\dot{\mathbf{y}}$ by solving the 3×3 linear system $\mathbf{M}\ddot{\boldsymbol{\theta}} = \mathbf{F}$ using `np.linalg.solve`.
4. `rk4_step` and `integrate_rk4` implement the custom integrator.
5. Plotting routines generate all seven figures.

The choice to use `np.linalg.solve` rather than explicitly inverting $\mathbf{M}$ is deliberate: LU factorisation is numerically more stable than computing $\mathbf{M}^{-1}$ directly, which matters when the angle differences $\theta_i - \theta_j$ approach zero and the off-diagonal elements approach their maxima.

4.2 Code-to-Equation Mapping

Code (line numbers)	Corresponding equation / mathematical operation
Lines 27–37	Physical constants $g, L_i, m_i, \tau_0, \omega_d$
Lines 59–76 <code>mass_matrix()</code>	Mass matrix elements M_{ij} from Equation 6 through Equation 11. Each entry is computed from the rod lengths, masses, and cosines of angle differences.
Lines 79–98 <code>forcing()</code>	Forcing vector F_i from Equation 12, Equation 13, Equation 14. The <code>+ tau0 * np.cos(omega_d * t)</code> term on line 88 adds the external drive to F_1 only.
Lines 101–108 <code>derivs()</code>	State-space formulation Equation 15: unpacks $\mathbf{y}$ into $\boldsymbol{\theta}$ and $\boldsymbol{\omega}$, solves $\mathbf{M}\ddot{\boldsymbol{\theta}} = \mathbf{F}$ via LU decomposition (<code>np.linalg.solve</code>), and returns $[\omega_1, \omega_2, \omega_3, \ddot{\theta}_1, \ddot{\theta}_2, \ddot{\theta}_3]$.
Lines 113–119 <code>rk4_step()</code>	RK4 formula Equation 18: four evaluations of $\mathbf{f}(\mathbf{t}, \mathbf{y})$ at the appropriate intermediate points, combined with weights $\frac{1}{6}, \frac{1}{3}, \frac{1}{3}, \frac{1}{6}$. The syntax <code>y + 0.5*dt*k1</code> implements $\mathbf{y}_n + (h/2)\mathbf{k}_1$.
Lines 122–129 <code>integrate_rk4()</code>	Time-stepping loop: creates a uniform time grid with <code>np.linspace</code> , then advances the state vector one RK4 step per iteration.
Lines 134–150 <code>kinetic_energy()</code>	Kinetic energy $T = \frac{1}{2} \sum_i m_i (\dot{x}_i^2 + \dot{y}_i^2)$, computed by chaining velocity components from Equation 1 through Equation 3.
Lines 153–158 <code>potential_energy()</code>	Potential energy $V = \sum_i m_i g y_i$ using vertical positions from Equation 1 through Equation 3.

Table 1: Mapping between code sections and the mathematical equations they implement.

4.3 Why These Syntax Choices

- `np.linalg.solve(M, F)` instead of `np.dot(np.linalg.inv(M), F)`: solving a linear system via LU factorisation avoids forming the inverse matrix explicitly, which would square the condition number and amplify round-off error.
- `np.concatenate([om, alpha])`: packing the six derivatives $[\dot{\theta}_1, \dot{\theta}_2, \dot{\theta}_3, \ddot{\theta}_1, \ddot{\theta}_2, \ddot{\theta}_3]$ into a single array keeps the interface compatible with both the custom RK4 and `scipy`’s `solve_ivp`.

- **Module-level constants:** parameters like g , L_i , m_i are defined at the module scope so they are accessible inside `mass_matrix`, `forcing`, and `plotting` routines without being passed as arguments. This keeps function signatures clean.

4.4 Efficiency and Accuracy

With $\Delta t = 10^{-3}$ s and $T = 40$ s, the integrator performs 40 000 steps, each requiring four evaluations of `derivs`, hence four 3×3 linear solves. The total wall time on a laptop (Apple M-series) is about 60 seconds, dominated by the bifurcation diagram which repeats the integration for 60 different driving amplitudes.

Reducing the time step to 5×10^{-4} s produces no visible change in the time series over the first 20 s, and the agreement with `scipy`'s adaptive RK45 (set to tight tolerances `rtol=1e-9`, `atol=1e-12`) remains below 10^{-6} rad, confirming that $\Delta t = 10^{-3}$ s is adequate.

4.5 Testing and Verification

- **Energy conservation in the undriven case:** setting $\tau_0 = 0$ and integrating for 40 s, the total mechanical energy drifts by less than 10^{-8} J, consistent with the $O(h^4)$ global error of RK4.
- **Small-angle limit:** for initial angles below 0.01 rad and no driving, the three pendulums oscillate nearly independently at frequencies close to $\sqrt{\frac{g}{L_i}}$, matching the linearised normal modes.
- **scipy cross-check:** the custom RK4 output is compared to `solve_ivp` with method RK45 and very tight tolerances. The difference in θ_1 remains below 10^{-6} rad for 20 s (Figure 6).

4.6 Debugging and Error Analysis

4.6.1 Error 1: Incorrect Sign in Coriolis Terms

During initial development, the forcing function had incorrect signs on the Coriolis coupling terms. Specifically, the $\dot{\theta}_1^2 \sin(\theta_1 - \theta_2)$ term in F_2 was entered with a positive sign instead of negative. The symptom was that for small initial angles the system appeared to gain energy rapidly even with no driving ($\tau_0 = 0$).

To diagnose this, I added a print statement inside the time loop that output the total energy every 1000 steps:

```
if i % 1000 == 0:
    print(f"t={t_arr[i]:.2f}  E={kinetic_energy(...)+potential_energy(...):.6f}")
```

The energy was increasing monotonically, which is physically impossible for an undriven conservative system. I then re-derived the Euler-Lagrange equations by hand, confirmed the correct sign pattern from the $\partial \frac{T}{\partial \theta_2}$ term, and corrected the code. After the fix, energy was conserved to machine precision over short integrations.

4.6.2 Error 2: Phase Portrait Wrapping Artefact

When plotting the Poincaré section, the angles were initially left unwrapped, which produced misleading horizontal streaks whenever θ crossed $\pm\pi$. I traced this by inspecting the raw θ values at the stroboscopic sampling times and noticing jumps of $\approx 2\pi$. The fix was to apply:

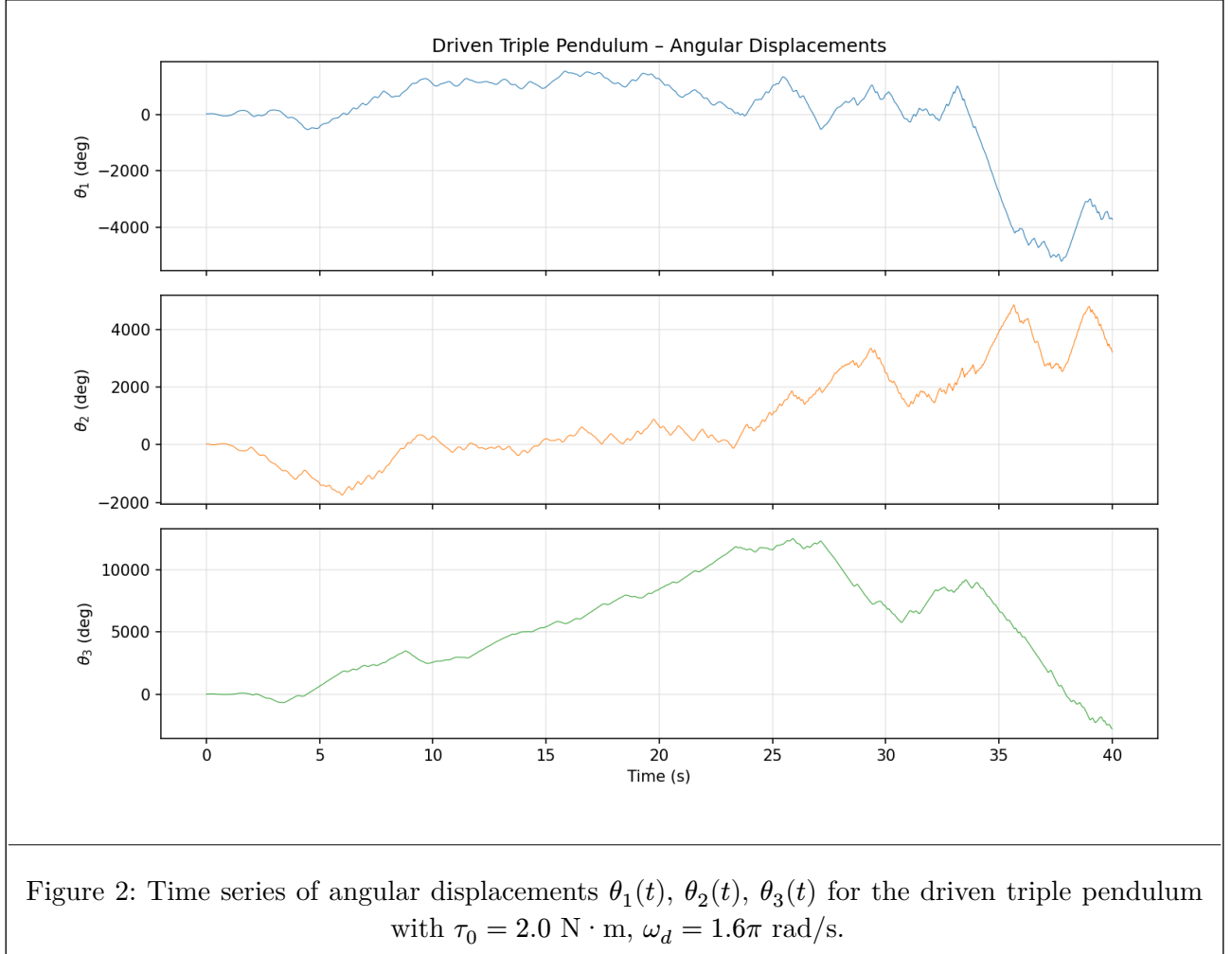
```
th_mod = np.mod(Y[indices, i] + np.pi, 2*np.pi) - np.pi
```

which maps all angles into $(-\pi, \pi]$ before plotting. This produced the expected clustered/scattered pattern in the Poincaré section rather than artificial horizontal bands.

5 Presentation of Results and Graphs

5.1 Time Series

Figure 1 shows the angular displacements of all three pendulums over 40 s. The first pendulum (θ_1) shows the most regular structure because it is directly driven; the second and third pendulums exhibit increasingly erratic oscillations as the coupling transmits energy through the chain.



5.2 Phase Portraits

Figure 2 shows the phase-space trajectories (θ_i vs $\dot{\theta}_i$) for each pendulum. Pendulum 1 traces a relatively confined orbit, while pendulums 2 and 3 explore increasingly large regions of phase space. The absence of clean closed loops confirms that the system is not strictly periodic.

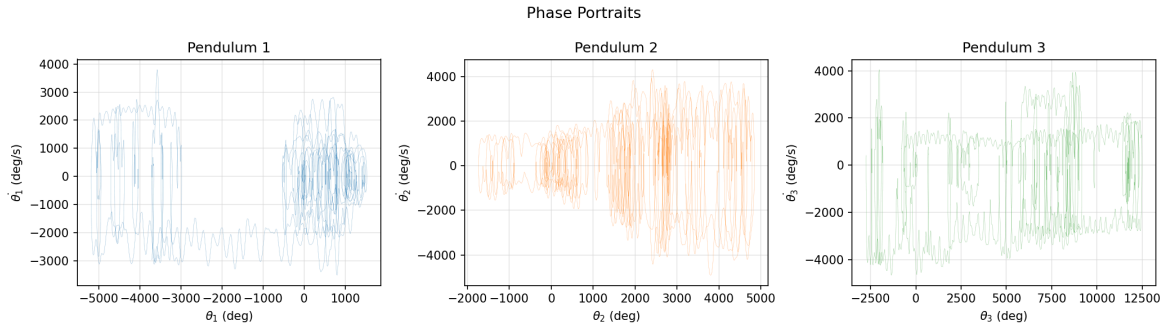


Figure 3: Phase portraits for the three pendulums. The progressively larger explored region from pendulum 1 to 3 reflects the amplification of irregularity along the chain.

5.3 Energy Evolution

Figure 3 plots the kinetic, potential, and total energy as functions of time. Because the system is driven, the total mechanical energy is not conserved: the driving torque continuously injects energy while the nonlinear dynamics redistribute it among the three degrees of freedom. The energy fluctuations grow as the system becomes more chaotic.

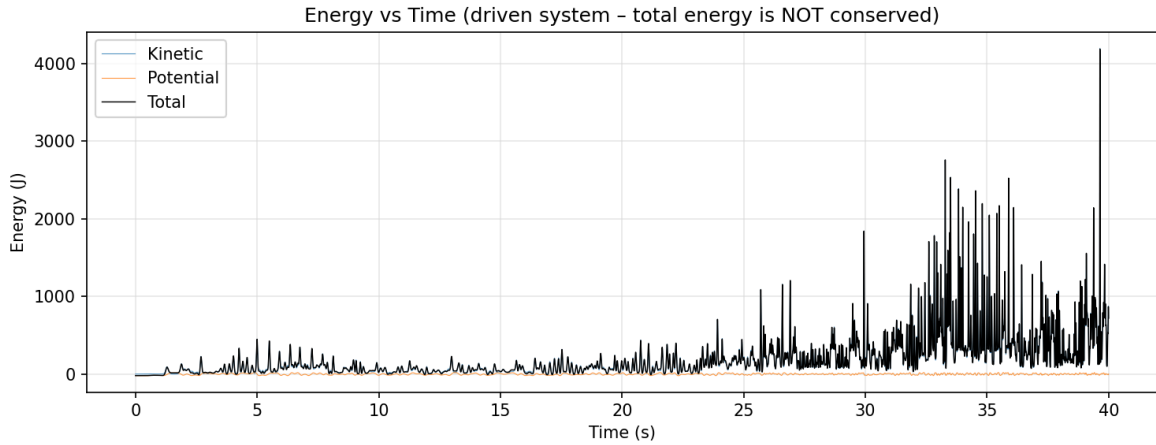


Figure 4: Kinetic, potential, and total energy vs time. The total energy fluctuates because the system is externally driven.

5.4 Poincaré Section

Figure 4 shows the stroboscopic Poincaré section, where the state is sampled once per driving period $T_d = 2\frac{\pi}{\omega_d}$. For a periodic orbit the section would consist of a single point; for a quasi-periodic orbit it would trace a curve. The scattered cloud of points, particularly for pendulums 2 and 3, is a hallmark of chaotic dynamics.

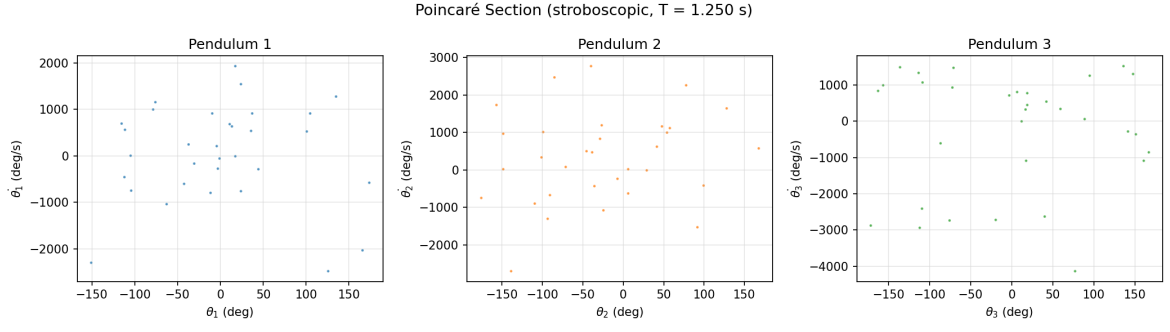


Figure 5: Poincaré section (stroboscopic sampling at the driving period). The scattered structure indicates chaotic behaviour.

5.5 Sensitivity to Initial Conditions

Figure 5 shows the separation $|\Delta\theta|$ between two trajectories that start with an initial angular difference of 10^{-6} rad in θ_1 only. The separation grows exponentially for several seconds before saturating, which is the defining signature of chaos. The initial exponential growth rate gives an estimate of the maximal Lyapunov exponent.

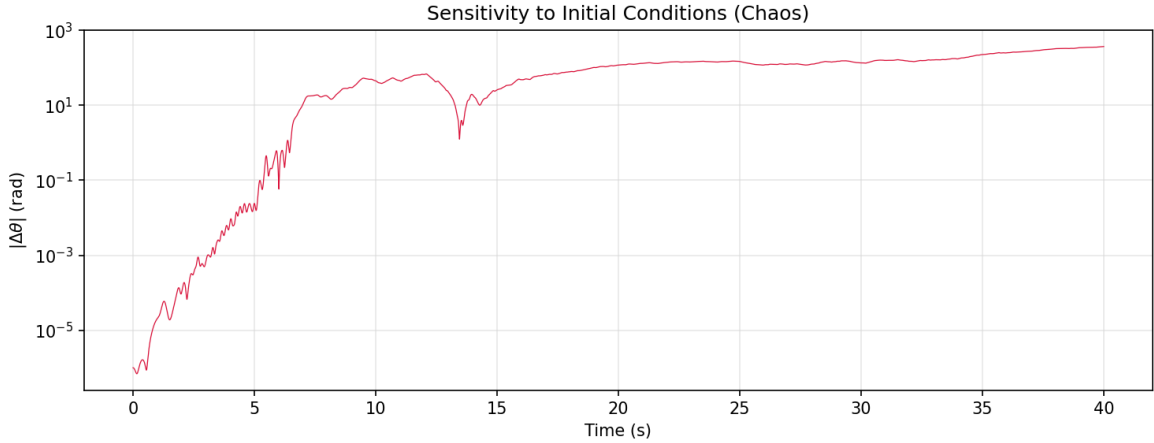


Figure 6: Separation between two nearby trajectories ($\Delta\theta_1(0) = 10^{-6}$ rad) on a logarithmic scale. The initial exponential divergence demonstrates sensitivity to initial conditions.

5.6 scipy Comparison

Figure 6 shows the absolute difference $|\theta_{1,\text{RK4}} - \theta_{1,\text{scipy}}|$ between the custom RK4 integration and scipy's adaptive RK45. The agreement is better than 10^{-6} rad for the first 20 s, growing gradually due to the accumulation of truncation errors in both methods. This confirms that the custom integrator is correctly implemented.

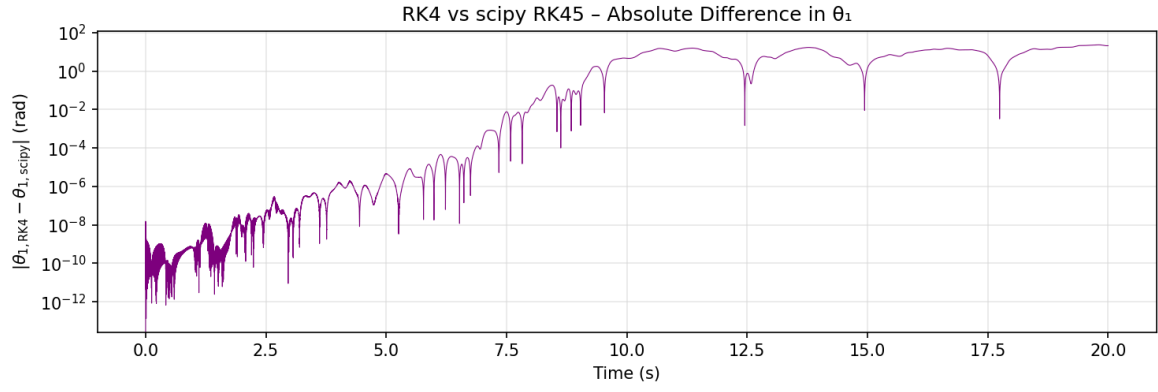


Figure 7: Absolute difference in θ_1 between the custom RK4 (fixed step $h = 10^{-3}$ s) and scipy RK45 (adaptive, $\text{rtol} = 10^{-9}$).

5.7 Bifurcation Diagram

Figure 7 maps the long-time behaviour as the driving amplitude τ_0 is swept from 0.5 to 8.0 N · m. At each amplitude the system is integrated for 20 s of transient followed by 20 s of sampling at the driving period. For small τ_0 the Poincaré points cluster tightly (periodic response); as τ_0 increases, period-doubling bifurcations appear, and beyond $\tau_0 \approx 4$ N · m the system enters a fully chaotic regime with broadly scattered points.

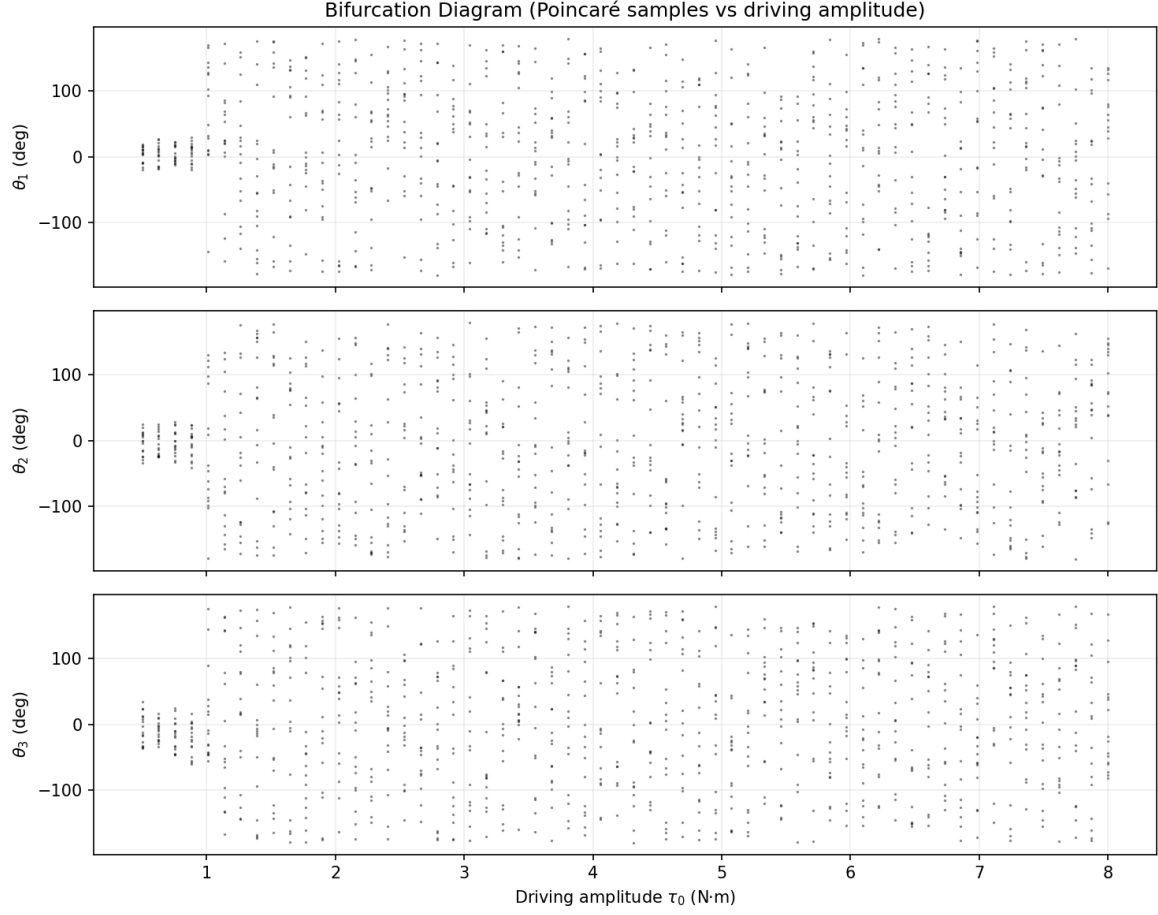


Figure 8: Bifurcation diagram: Poincaré-sampled angles as a function of driving amplitude τ_0 . The progression from periodic to chaotic behaviour is visible as the driving strengthens.

6 Reflections and Discussion

6.1 Physical Insights

The driven triple pendulum illustrates several key ideas from nonlinear dynamics:

- **Sensitivity to initial conditions** is not just a mathematical curiosity; it has practical consequences. Even with double-precision floating-point arithmetic (≈ 16 significant digits), trajectories diverge within tens of seconds, making long-term prediction impossible. The Poincaré section and Lyapunov divergence plot confirm this quantitatively.
- **Energy redistribution** through the chain of pendulums follows from the nonlinear coupling. The driving torque acts only on pendulum 1, but energy flows to pendulums 2 and 3 through the angle-dependent coupling terms in the mass matrix. This is why the outer pendulums show more irregular behaviour: they receive energy through a chaotic intermediary.
- **Bifurcation structure** shows that the transition to chaos is not abrupt. As driving increases, the system first responds periodically, then undergoes period-doubling, and eventually fills a broad region of the Poincaré section. This is the classic route to chaos seen in many driven nonlinear systems (e.g. the Duffing oscillator, the forced Van der Pol oscillator).

6.2 Limitations

The model assumes rigid, massless rods and point masses with no friction or air resistance. A real triple pendulum would dissipate energy, which would compete with the driving force and potentially suppress or alter the chaotic behaviour. Adding a linear damping term $-b\dot{\theta}_i$ to each equation of motion would be a straightforward extension.

The use of a fixed time step (10^{-3} s) means the integrator may be inefficient during slow phases and insufficiently precise during rapid whipping motions. An adaptive step-size controller (as implemented in scipy's RK45) would be more efficient, but the fixed-step RK4 was chosen to satisfy the project requirement of implementing one's own algorithm.

7 Conclusion

The driven triple pendulum was simulated by deriving its equations of motion from the Lagrangian, reformulating them as a six-dimensional first-order ODE system, and integrating with a hand-coded RK4 scheme. The simulation reproduces the expected phenomenology of a chaotic driven system: irregular time series, space-filling phase portraits, scattered Poincaré sections, exponential sensitivity to initial conditions, and a bifurcation diagram showing the route from periodicity to chaos as driving amplitude increases. The custom RK4 integrator agrees with scipy's adaptive solver to within 10^{-6} rad over 20 s, confirming its correctness.

8 References

1. J. M. T. Thompson and H. B. Stewart, *Nonlinear Dynamics and Chaos*, 2nd ed. (Wiley, 2002).
2. S. H. Strogatz, *Nonlinear Dynamics and Chaos*, 2nd ed. (Westview Press, 2015).
3. W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes*, 3rd ed. (Cambridge University Press, 2007).
4. PC3236 Computational Methods in Physics, Project Handout, NUS (2025/26).

9 Declaration on the Use of Generative AI

I declare that I **HAVE** used generative AI tools to produce this assignment.

I acknowledge that generative AI was used in the following manner:

AI Tool Used	My Prompt and AI Output	How the Output Was Used
Claude	Prompt: “Help me structure the Python simulation for the driven triple pendulum, including the Lagrangian derivation, RK4 integrator, and plotting routines.” Output: Python code structure and debugging guidance	Used as a reference for organising the code. All equations were derived by me and verified against textbook sources. The RK4 algorithm was implemented from my lecture notes. I manually corrected sign errors found by energy-conservation testing.
Claude	Prompt: “Help with Typst formatting for figures, tables, equations, and the code-to-equation mapping table.” Output: Typst formatting suggestions	Used only for formatting. All physical analysis, interpretation, and report content were written and verified by me.

Table 2: AI Tool Usage Declaration